FONCTIONS (MÉTHODES)

1. Définition :

Une **fonction** (ou méthode) est un **bloc de code réutilisable** qui effectue une tâche spécifique.

- Elle peut **prendre des paramètres** (entrées).
- Elle peut **retourner une valeur** ou ne rien retourner (void).

2. Syntaxe générale :

```
[modificateur] typeRetour NomFonction(paramètres)
{
    // Instructions
    return valeur; // si typeRetour n'est pas void
}
```

Explication :

- modificateur : public, private, etc. (optionnel pour l'exemple simple)
- typeRetour : type de la valeur retournée (int, string, bool, void si rien n'est retourné)
- NomFonction : nom de la fonction (camelCase ou PascalCase)
- paramètres : valeurs que la fonction peut recevoir

3. Exemple 1 : Fonction sans paramètre et sans retour (void) :

```
using System;

class Program{
    static void Saluer(){
        Console.WriteLine("Bonjour !");
    }
    static void Main(){
        Saluer(); // Appel de la fonction
    }
}
```

Explication :

- void → la fonction ne renvoie rien
- Saluer() → exécute simplement le bloc de code

4. Exemple 2 : Fonction avec paramètres :

```
static void SaluerPersonne(string nom){
    Console.WriteLine("Bonjour " + nom + " !");
}
static void Main(){
    SaluerPersonne("Alice");
    SaluerPersonne("Bob");
```

```
}
```

Explication :

- string nom → paramètre reçu par la fonction
- Permet de personnaliser le message pour chaque appel

5. Exemple 3 : Fonction avec retour:

```
static int Addition(int a, int b){
    return a + b;
}
static void Main(){
    int resultat = Addition(5, 3);
    Console.WriteLine("Résultat : " + resultat);
}
```

Explication :

- int → type de retour de la fonction
- return a + b; → renvoie la somme à l’appelant